SQL JOINs Practice Tasks

Oracle HR Schema

Group	Tank54
Schema	Oracle HR
Topics	Self Join, Inner Join, Left Join, Right Join, Full Join, Multi-Table Joins
Techniques	WHERE, ORDER BY, CASE, GROUP BY, HAVING, COUNT, AVG, SUM, MIN, MAX
Total Tasks	24

Table of Contents

Schema Overview 3

Section 1: Self Join 3

Section 2: Inner Join 4

Section 3: Left Join 5

Section 4: Right Join 6

Section 5: Full Join 7

Section 6: Multi-Table Joins (Combined Challenge) 8

Schema Overview

This practice document uses the Oracle HR schema. The main tables you will work with are: **employees** (employee_id, first_name, last_name, email, phone_number, hire_date, job_id, salary, commission_pct, manager_id, department_id), **departments** (department_id, department_name, manager_id, location_id), **jobs** (job_id, job_title, min_salary, max_salary), **locations** (location_id, street_address, postal_code, city, state_province, country_id), **countries** (country_id, country_name, region_id), and **regions** (region_id, region_name). Use the SQL Simulator to write and test your queries.

Each section focuses on a specific JOIN type. The tasks start simple and become more difficult. Later tasks combine multiple techniques such as CASE expressions, aggregate functions (COUNT, AVG, SUM, MIN, MAX), GROUP BY, HAVING, WHERE conditions, and ORDER BY clauses. Read the hint for each task to understand which tables and techniques you need. Write your SQL query in the answer area and test it in the simulator.

Table Reference

Table	Key Columns	Description
employees	employee_id, first_name, last_name, job_id, salary, manager_id, department_id, hire_date	All employee records with personal info, job, salary, and department
departments	department_id, department_name, manager_id, location_id	Company departments with location reference
jobs	job_id, job_title, min_salary, max_salary	Job roles and their salary ranges
locations	location_id, street_address, city, state_province, country_id	Office locations across the world
countries	country_id, country_name, region_id	Countries grouped by region
regions	region_id, region_name	Geographic regions (Americas, Europe, Asia, etc.)

Section 1: Self Join

A self join is when you join a table with itself. You use table aliases (e.g., e1 and e2) to distinguish between the two copies. This is useful for finding relationships between rows in the same table, such as employees and their managers.

Task 1. Find the name, salary, and hire date of all employees who work in the same department as 'Neena Kochhar'. Show only employees who were hired after Neena Kochhar.

Techniques: Self Join, WHERE

Hint: Tables: employees (aliased as e1, e2) | Join on department_id | WHERE e2.first_name = 'Neena' AND e2.last_name = 'Kochhar' | Filter: e1.hire_date > e2.hire_date

Task 2. List each employee name with their manager name and the manager salary. Show only employees whose manager earns more than 12000.

Techniques: Self Join, WHERE

Hint: Tables: employees e1 JOIN employees e2 ON e1.manager_id = e2.employee_id | WHERE e2.salary > 12000

Task 3. Find pairs of employees who have the same job_id. Show employee A name, employee B name, and the job_id. Exclude pairs where employee A and B are the same person. Count how many pairs exist per job_id.

Techniques: Self Join, GROUP BY, COUNT, WHERE

Hint: Tables: employees e1 JOIN employees e2 ON e1.job_id = e2.job_id AND e1.employee_id < e2.employee_id | GROUP BY e1.job_id | COUNT(*)

Task 4. Find all employees who earn more than their manager. Show employee name, employee salary, manager name, and manager salary. Order the results by the difference in salary from largest to smallest.

Techniques: Self Join, WHERE, ORDER BY

Hint: Tables: employees e1 JOIN employees e2 ON e1.manager_id = e2.employee_id | WHERE e1.salary > e2.salary | ORDER BY (e1.salary - e2.salary) DESC

Section 2: Inner Join

An inner join returns only rows that have matching values in both tables. It is the most common type of join. If a row in the left table has no match in the right table, that row is not included in the result.

Task 5. Show the first name, last name, job title, and department name for all employees who work in departments located in 'Seattle'.

Techniques: Inner Join, WHERE

Hint: Tables: employees e INNER JOIN departments d ON e.department_id = d.department_id JOIN locations l ON d.location_id = l.location_id | WHERE l.city = 'Seattle'

Task 6. Find the total salary, average salary, and number of employees for each department. Show only departments where the average salary is greater than 9000. Display department name alongside the results.

Techniques: Inner Join, GROUP BY, HAVING, AVG, SUM, COUNT

Hint: Tables: employees e INNER JOIN departments d ON e.department_id = d.department_id | GROUP BY d.department_name | HAVING AVG(e.salary) > 9000 | Functions: SUM, AVG, COUNT

Task 7. List all employees whose salary is higher than the average salary of their department. Show employee name, salary, department name, and the average salary of that department. Order by salary from highest to lowest.

Techniques: Inner Join, Subquery, AVG, WHERE, ORDER BY

Hint: Tables: employees e INNER JOIN departments d ON e.department_id = d.department_id | Subquery to find AVG salary per department | WHERE e.salary > (subquery)

Task 8. For each job title, show the job title, the number of employees in that job, the minimum salary, and the maximum salary. Show only jobs that have at least 3 employees.

Techniques: Inner Join, GROUP BY, HAVING, COUNT, MIN, MAX

Hint: Tables: employees e INNER JOIN jobs j ON e.job_id = j.job_id | GROUP BY j.job_title | HAVING COUNT(*) >= 3 | Functions: COUNT, MIN, MAX

Section 3: Left Join

A left join (or left outer join) returns all rows from the left table, and the matched rows from the right table. If there is no match, the right side columns will contain NULL. This is useful when you want to see all records from one table regardless of whether they have related records in another table.

Task 9. List all departments and the number of employees in each department. Include departments that have zero employees. Show department name and employee count.

Techniques: Left Join, GROUP BY, COUNT

Hint: Tables: departments d LEFT JOIN employees e ON d.department_id = e.department_id | GROUP BY d.department_name | COUNT(e.employee_id)

Task 10. Find all employees who do NOT have a department assigned (department_id is NULL). Show their first name, last name, email, and hire date.

Techniques: Left Join, WHERE, IS NULL

Hint: Tables: employees e LEFT JOIN departments d ON e.department_id = d.department_id | WHERE d.department_id IS NULL

Task 11. List all job titles and for each job, show the number of employees. Include jobs that currently have no employees. Order by the employee count from most to least.

Techniques: Left Join, GROUP BY, COUNT, ORDER BY

Hint: Tables: jobs j LEFT JOIN employees e ON j.job_id = e.job_id | GROUP BY j.job_title | COUNT(e.employee_id) | ORDER BY COUNT(e.employee_id) DESC

Task 12. Show each country name and the number of departments located in that country. Include countries that have zero departments. Use CASE to label countries with zero departments as 'No Departments'.

Techniques: Left Join, GROUP BY, COUNT, CASE

Hint: Tables: countries c LEFT JOIN locations l ON c.country_id = l.country_id LEFT JOIN departments d ON l.location_id = d.location_id | GROUP BY c.country_name | COUNT(d.department_id) | CASE WHEN COUNT = 0 THEN 'No Departments' END

Section 4: Right Join

A right join (or right outer join) returns all rows from the right table, and the matched rows from the left table. If there is no match, the left side columns will contain NULL. In Oracle, RIGHT JOIN works the same as RIGHT OUTER JOIN.

Task 13. List all departments and any employees who were hired in the year 2005 or later. Show department name, employee name, and hire date. Include departments that had no new hires since 2005.

Techniques: Right Join, WHERE, OR, IS NULL

Hint: Tables: employees e RIGHT JOIN departments d ON e.department_id = d.department_id | WHERE e.hire_date >= '2005-01-01' OR e.hire_date IS NULL

Task 14. Show all locations and the employees who work in those locations. Include locations that have no employees assigned. Display city, street address, and employee name.

Techniques: Right Join, WHERE

Hint: Tables: employees e RIGHT JOIN departments d ON e.department_id = d.department_id RIGHT JOIN locations l ON d.location_id = l.location_id | OR use LEFT JOIN from locations side

Task 15. Find all regions and the number of employees in each region. Include regions that have zero employees. Display region name and employee count.

Techniques: Right Join (multi-table), GROUP BY, COUNT

Hint: Tables: employees e RIGHT JOIN departments d ON e.department_id = d.department_id RIGHT JOIN locations l ON d.location_id = l.location_id RIGHT JOIN countries c ON l.country_id = c.country_id RIGHT JOIN regions r ON c.region_id = r.region_id | GROUP BY r.region_name | COUNT(e.employee_id)

Task 16. Show each job title and the employees working in that job. Include jobs with no employees. Use CASE to show 'Vacant Position' for jobs without employees. Order by whether the job is filled or not.

Techniques: Right Join (or Left Join), CASE, ORDER BY

Hint: Tables: jobs j LEFT JOIN employees e ON j.job_id = e.job_id | SELECT j.job_title, CASE WHEN e.employee_id IS NULL THEN 'Vacant Position' ELSE e.first_name || ' ' || e.last_name END | ORDER BY CASE WHEN e.employee_id IS NULL THEN 1 ELSE 0 END

Section 5: Full Join

A full join (or full outer join) returns all rows from both tables. Where there is a match, the rows are combined. Where there is no match, NULL values appear on the side that has no data. This is useful when you want a complete picture of data from both tables.

Task 17. List all employees and all departments. Show employee name and department name. For employees without a department, show 'No Department'. For departments without employees, show 'No Employees'.

Techniques: Full Join, CASE

Hint: Tables: employees e FULL JOIN departments d ON e.department_id = d.department_id | CASE WHEN e.employee_id IS NULL THEN 'No Employees' ELSE e.first_name || ' ' || e.last_name END | Similar CASE for department

Task 18. Show all employees and all jobs. Display employee name and job title. Show 'Unassigned Job' for employees without a job match and 'No Employee' for jobs without any employee. Count total rows in the result.

Techniques: Full Join, CASE

Hint: Tables: employees e FULL JOIN jobs j ON e.job_id = j.job_id | CASE expressions for NULL values on both sides

Task 19. Combine all departments and all locations using a full join. Show department name, city, and country. Label missing departments as 'No Department in This Location' and missing locations as 'Location Not Set'.

Techniques: Full Join, CASE, Left Join

Hint: Tables: departments d FULL JOIN locations l ON d.location_id = l.location_id LEFT JOIN countries c ON l.country_id = c.country_id | CASE expressions for both NULL sides

Task 20. Find the total number of employees per country using a full join. Include countries with zero employees. Show country name, total employees, and a CASE label: 'Active' if employees > 0, 'No Staff' if zero.

Techniques: Full Join (multi-table), GROUP BY, COUNT, CASE

Hint: Tables: employees e FULL JOIN departments d ON e.department_id = d.department_id FULL JOIN locations l ON d.location_id = l.location_id FULL JOIN countries c ON l.country_id = c.country_id | GROUP BY c.country_name | COUNT, CASE

Section 6: Multi-Table Joins (Combined Challenge)

In real business scenarios, you often need data from three or more tables at the same time. Multi-table joins let you connect employees, departments, locations, jobs, and other tables in a single query. These tasks combine different join types and techniques.

Task 21. Show the full employee chain: employee name, their manager name, manager's job title, and the department name for both employee and manager. Show only employees whose manager works in a different department.

Techniques: Self Join + Inner Join, WHERE

Hint: Tables: employees e1 JOIN employees e2 ON e1.manager_id = e2.employee_id JOIN departments d1 ON e1.department_id = d1.department_id JOIN departments d2 ON e2.department_id = d2.department_id JOIN jobs j ON e2.job_id = j.job_id | WHERE d1.department_id != d2.department_id

Task 22. For each department, show the department name, the number of employees, the average salary, and the city where the department is located. Show only departments located in the 'Americas' region. Order by average salary from highest to lowest.

Techniques: Multi-table Inner Join, GROUP BY, WHERE, ORDER BY, AVG, COUNT

Hint: Tables: employees e JOIN departments d ON e.department_id = d.department_id JOIN locations l ON d.location_id = l.location_id JOIN countries c ON l.country_id = c.country_id JOIN regions r ON c.region_id = r.region_id | WHERE r.region_name = 'Americas' | GROUP BY d.department_name, l.city | AVG, COUNT, ORDER BY

Task 23. List all countries and show: the number of departments, the number of employees, and the number of locations in each country. Include countries that have zero departments, zero employees, or zero locations. Use CASE to label each count.

Techniques: Left Join (multi-table), GROUP BY, COUNT(DISTINCT), CASE

Hint: Tables: countries c LEFT JOIN locations l ON c.country_id = l.country_id LEFT JOIN departments d ON l.location_id = d.location_id LEFT JOIN employees e ON d.department_id = e.department_id | GROUP BY c.country_name | COUNT(DISTINCT d.department_id), COUNT(DISTINCT l.location_id), COUNT(e.employee_id) | CASE

Task 24. Create a salary analysis report by job title and department: show job title, department name, total salary paid, number of employees, and the ratio of total salary to number of employees (average salary). Use CASE to classify: 'High Budget' if total > 50000, 'Medium Budget' if total > 20000, 'Low Budget' otherwise. Order by total salary descending.

Techniques: Inner Join, GROUP BY, SUM, COUNT, CASE, ORDER BY

Hint: Tables: employees e JOIN jobs j ON e.job_id = j.job_id JOIN departments d ON e.department_id = d.department_id | GROUP BY j.job_title, d.department_name | SUM(e.salary), COUNT(e.employee_id), SUM/COUNT | CASE WHEN SUM > 50000 THEN 'High Budget' WHEN SUM > 20000 THEN 'Medium Budget' ELSE 'Low Budget' END | ORDER BY SUM DESC